Feature Scaling: Standardization

Module 24 Study Notes • Feature Engineering

1. What is Feature Scaling?

In many datasets, input features exist on wildly different scales (e.g., Age ranges from 0-100, while Salary ranges from 10,000-1,000,000). Machine Learning algorithms that rely on distance calculations or gradient-based optimization struggle with these discrepancies. **Feature Scaling** is the process of bringing all numerical features into a common range to ensure stability and accuracy.

2. Why is Scaling Necessary?

Consider an algorithm like K-Nearest Neighbors (KNN), which calculates the distance between data points. If one feature (Salary) has much larger values than another (Age), the Salary column will dominate the mathematical distance, effectively rendering the Age feature invisible to the model.

3. Standardization (Z-Score Normalization)

Standardization is the most common scaling technique. It transforms data such that it has a **Mean of 0** and a **Standard Deviation of 1**.

The Mathematical Formula:

> $z = (x - \mu) / \sigma$
>
> - x: The original data point.
> - μ *(mu)*: The mean of the column.
> - σ *(sigma)*: The standard deviation of the column.

4. Implementation Best Practices

- **Split First:** Always perform a `train_test_split` *before* scaling.

- **Fit on Train:** Call `scaler.fit()` and `scaler.transform()` on the **Training Set** only. This learns the mean and standard deviation from the training data.
- **Transform Test:** Call only `scaler.transform()` on the **Test Set**. Do *not* re-fit on the test set, as this causes data leakage (the model would "see" information about the test set's distribution during training).

5. Impact on Outliers

A crucial realization is that **Standardization does NOT fix outliers**. Outliers will still be present in the transformed data, and they will influence the mean and standard deviation used for scaling, potentially causing the rest of your data to be squeezed into a very narrow range.

6. When to Use Standardization?

Standardization is mandatory for algorithms that involve:

- **Distance-based algorithms:** KNN, K-Means Clustering, Support Vector Machines (SVM).
- **Gradient-based optimization:** Linear Regression, Logistic Regression, and all Neural Networks (Deep Learning). Without scaling, these models often fail to converge (find the optimal weights).

Algorithms like **Decision Trees, Random Forests, and Gradient Boosting (XGBoost/LightGBM)** are generally invariant to feature scaling, as they make splits based on relative comparisons rather than distances or gradients.